

Version 1.0.3

Installation

ContextMenu.js depends on two external scripts: **jQuery** & **jQueryUI**. If you don't want to include all of jQuery UI's widgets, at the bare minimum you will need to include **jQueryUI Core** and **jQueryUI position**. If you want to have icons you also need to include **FontAwesome**. Here is a basic document structure to get you started:

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>ContextMenu.js Demo</title>
5     <link href="https://maxcdn.bootstrapcdn.com/font-awesome/4.4.0/css/font-aw
      ="stylesheet" />
6     <link href="contextmenu.min.css" rel="stylesheet" />
7     <script src="https://code.jquery.com/jquery-2.1.4.min.js"></script>
8     <script src="https://code.jquery.com/ui/1.11.4/jquery-ui.min.js"></script>
9     <script src="contextmenu.min.js"></script>
10  </head>
11  <body>
12    <br />
13    <div id="contextmenu"></div>
14    <p>
15      </body>
16  </html>
```

And then, attach a drop down menu to an element using jQuery:

```
1 $('#contextmenu').contextMenu({
2   items: [
3     {type: 'item', text: 'Cut', icon: 'fa fa-scissors'},
4     {type: 'item', text: 'Copy', icon: 'fa fa-files-o'},
5     {type: 'item', text: 'Paste', icon: 'fa fa-clipboard'},
6     {type: 'divider'},
7     {type: 'item', text: 'Delete', icon: 'fa fa-times'}
8   ]
9 });
```

Menu Events

ContextMenu.js offers much more than just context menus. It can also be used to create drop down lists, navigations, and more. To achieve that, menus can be triggered by 5 different events: `click`, `dblclick`, `hover`, `focus`, and the default `contextmenu`. To set the triggering event, use the `event` option.

For the `hover` event, it is recommended to also set `autoHide` to a value different than the default 0, since the menu will be automatically hidden when the mouse leaves the element. Read more about automatic hiding below.

```

1 $('#contextmenu').contextMenu({
2     event: 'click',
3     items: [ ... ]
4 });
5
6 // Setting the event to 'hover' causes the menu to automatically hide immediately
7 $('#contextmenu').contextMenu({
8     autoHide: 300, // Wait 300ms before hiding the menu
9     event: 'hover',
10    items: [ ... ]
11 });

```

Dynamic Event Binding

Context menus can be bound to dynamically created elements using the `selector` option. Simply call jQuery on an existing element (one that exists at the time of the call) and provide a selector string to the `selector` option for the dynamic element(s).

```

1 // Select any element that exists at the time of the call
2 $('html').contextMenu({
3     // Selector string pointing to the dynamic element
4     selector: '.dynamic-child',
5     items: [ ... ]
6 });

```

Transition Effects

There are 8 transition effects that can be used to show/hide the menu. Available transitions are `slideTop`, `slideBottom`, `slideLeft`, `slideRight`, `flipHorizontal`, `flipVertical`, `fadeIn`, and the default `none`. It is also possible to set the speed of the transition in milliseconds using the `speed` argument (defaults to 0).

```

1 $('#contextmenu').contextMenu({
2     transition {
3         speed: 300, // In milliseconds
4         type: 'fadeIn'
5     },
6     items: [ ... ]
7 });

```

Menu Item Types

Menus can be constructed using one or more of the 4 available menu item types: `divider`, `title`, `item`, `checkbox`. The `items` option accepts an array of objects, each of which has a `type` key specifying the type of the item.

```

1 $('#contextmenu').contextMenu({
2     items: [
3         {type: 'title', text: 'File Controls'},
4         {type: 'item', text: 'Cut', icon: 'fa fa-scissors'},
5         {type: 'item', text: 'Copy', icon: 'fa fa-files-o'},
6         {type: 'item', text: 'Paste', icon: 'fa fa-clipboard'},
7         {type: 'divider'},
8         {type: 'item', text: 'Delete', icon: 'fa fa-times'}
9     ]
10 });

```

Divider

The divider component is a simple line used to divide the menu into different sections. It accepts no arguments.

```
1 {type: 'divider'}
```

Title

The title component is used to display static text inside the menu. It accepts the following arguments:

- `text` (*string*) Text/HTML to display as the title.

```
1 {type: 'title', text: 'Title Text Goes Here'}
```

Item

The item component is used to create menu items that can have events bounded to them and have sub-menus. It accepts the following arguments:

- `text` (*string*) Text/HTML to display as the item's text.
- `icon` (*string*) A FontAwesome CSS class to use as the icon (or a custom CSS class for custom icons).
- `disabled` (*boolean*) Set to `true` to disable the item. defaults to `false`.
- `click` (*function*) A function to call when the item is clicked. Takes the current event as an argument.
- `hover` (*function*) A function to call when the item is hovered over. Takes the current event as an argument.
- `id` (*string*) The item's element id. Leave blank for no id.

```
1 {  
2   type: 'item',  
3   text: 'Paste',  
4   icon: 'fa fa-clipboard',  
5   disabled: false,  
6   click: function(e) {}  
7   hover: function(e) {}  
8 }
```

Checkbox

- `text` (*string*) Text/HTML to display as the checkbox's text.
- `icon` (*string*) A FontAwesome CSS class to use as the icon (or a custom CSS class for custom icons).
- `disabled` (*boolean*) Set to `true` to disable the checkbox. defaults to `false`.
- `checked` (*boolean*) The initial checkbox value. Set to `true` to mark the checkbox as checked. defaults to `false`.
- `click` (*function*) A function to call when the checkbox is clicked (called after the checkbox is being toggled). Takes the current event as an argument.
- `hover` (*function*) A function to call when the checkbox is hovered over. Takes the current event as an argument.

- `id` (string) The item's element id. Leave blank for no id.

```
1 {  
2   type: 'checkbox',  
3   text: 'Checkbox Text',  
4   icon: '', // Used in addition to the checkmark icon  
5   disabled: false,  
6   checked: false,  
7   click: function(e) {}  
8   hover: function(e) {}  
9 }
```

Item Events

Menu-Wide Item Events

Event listeners can be assigned to all menu items by using the `click` or `hover` options. The functions accepts the event's data object as an argument. The `this` variable can be used to retrieve the calling item's settings.

Note menu-wide events will automatically be assigned to all item/checkbox types. To distinguish between the, use `this.type === 'item'` or `this.type === 'checkbox'`

```
1 $('#contextmenu').contextMenu({  
2   click: function(e) {  
3     // 'this' refers to the clicked item  
4     alert('The item "'+this.text+'" was clicked');  
5   },  
6   items: [  
7     {type: 'item', text: 'Item 1'},  
8     {type: 'item', text: 'Item 2'},  
9     {type: 'item', text: 'Item 3'}  
10  ]  
11 });
```

Item-Specific Events

Item event listeners can be set on a per-item basis. Item events are only applicable to item/checkbox types.

An item-specific event overrides a menu-wide event of the same type.

```

1 $('#contextmenu').contextMenu({
2     click: function(e) {
3         // 'this' refers to the clicked item
4         alert('The item "' + this.text + '" was clicked');
5     },
6     items: [
7         {type: 'item', text: 'Item 1'},
8         {type: 'item', text: 'Item 2'},
9         {type: 'item', text: 'Item 3', click: function(e){alert('Click event overr
10    ]
11 });

```

Dynamic Items

Menus can be dynamically constructed each time a selected event has been triggered. This can be used to create **event-specific menus** on the fly. The menus are destroyed and reconstructed every time.

To dynamically create menu items, set the `items` option as a function instead of an array. The function should return an array of menu items and take the current event as an argument.

```

1 $('#contextmenu').contextMenu({
2     items: function(e) {
3         return [
4             /* List of items */
5         ]
6     }
7 });

```

Positioning

Drop down menus can be positioned relative to the **target element**. The syntax used is similar to the one used by `jQuery.position()` (<http://api.jqueryui.com/position/>). For example, `{my: 'left top', at: 'center bottom'}` will position the menu's top left corner at the bottom and center of the target element. Additionally, by setting `at: 'mouse'` the menu can be positioned according to the **mouse coordinates**.

If you want the menu to be repositioned when clicking on certain children of the target element, you can do that using the `children` argument that takes a selector string.

Notice that the menu will automatically be **placed inside the screen** if it collides with the window's boundaries.

```

1 $('#contextmenu').contextMenu({
2     position: {
3         my: 'left top+5', // add 5px to the menu's top position
4         at: 'center bottom',
5         children: '.target-element-child'
6     },
7     items: [ ... ]
8 });

```

Action Hooks

There are 3 actions that can be hooked to for running custom functions at specific times.

- `show` – Triggered when the menu becomes visible. If another click is made on the target element, the menu will be repositioned, but the `show` hook will not be triggered again.
- `position` – Triggered when the menu becomes visible, and every time it is repositioned. For example, when the menu is visible and another click on the target element is made, the menu will be repositioned and the `position` hook will be triggered.
- `hide` – Triggered when the menu becomes hidden (after the transition effect has been completed).

```
1 $('#contextmenu').contextMenu({
2   items: [ ... ],
3   hooks: {
4     // Called when the menu is shown
5     show: function(e) {},
6
7     // Called when the menu is repositioned (also called every time the menu is
8     position: function(e){},
9
10    // Called when the menu is hidden
11    hide: function(e){}
12  }
13 });
```

Methods

There are several methods that can be called after a context menu has been instantiated.

show

Opens the menu. Accepts the current event as an argument.

```
1 $('#myElement').click(function(e){
2   $('#contextmenu').contextMenu('open',e);
3 });
```

hide

Closes the current menu. Accepts the current event as an argument.

```
1 $('#myElement').click(function(e){
2   $('#contextmenu').contextMenu('close',e);
3 });
```

refresh

Refreshes the contents of the menu (as if the menu has been closed and reopened). applicable if the menu items are being generated dynamically.

```
1 $('#myElement').click(function(e){
2   $('#contextmenu').contextMenu('refresh',e);
3 });
```

isVisible

Returns true if the menu is currently open.

```
1 $('#contextmenu').contextMenu('isVisible');
```

select

Select a menu item by its ID (item ID, not HTML ID).

```
1 // Select the first menu item
2 $('#contextmenu').contextMenu('select',0);
```

Auto Hiding

Menus can be set to automatically hide when the cursor moves out of them by setting the `autoHide` option to any number representing the delay in milliseconds. The default value of `autoHide` is `false`, disabling automatic hiding.

▲ Note that menus that use `hover` as their triggering event are automatically hidden by default.

```
1 $('#contextmenu').contextMenu({
2     autoHide: 500, // Time in milliseconds,
3                 // or 'false' to disable auto hiding
4     items: [ ... ]
5 });
```

Menu Arrow

The menu arrow is a little triangle used for pointing towards the mouse position or the target element. The arrow's position can be set by using the `arrow` option. possible values are `'auto'`, `'top'`, `'bottom'`, `'left'`, `'right'` or `false` for no arrow. Defaults to `'auto'`.

```
1 $('#contextmenu').contextMenu({
2     arrow: 'auto', // Or bottom, left, right, top (false for no arrow)
3     items: [ ... ]
4 });
```